ComicCraft - AI Comic Story Creator using Gemini Models

[redacted]

Project Description

ComicCraft is a web-based application that uses AI to generate personalized comic book stories and illustrations from user-provided prompts. It is built with FastAPI and integrates Google's Gemini models for text with Hugging Face's FLUX.1-schnell model for images, so the storyline, dialogue and comic-style artwork are produced automatically.

The application takes a story prompt, main character name, setting, tone and art style, and generates a five-panel storyline with a matching illustration for each panel. Users view the comic preview in the browser and download the full comic as a PDF containing images and narration.

A clean HTML interface rendered with Jinja2 templates keeps each story cohesive and visually engaging. The app serves a browser frontend, so comic creation is accessible to non-artists and casual users.

Scenarios

Scenario 1: A user enters a prompt describing their comic, such as "A brave fox exploring an enchanted forest." They also set the main character's name (e.g., Luna), the setting (e.g., Forest), the tone (e.g., Dramatic) and the art style (e.g., Anime). The system uses Gemini to plan and write the story and FLUX.1-schnell to draw each panel, producing a complete five-panel comic that fits the user's creative vision.

Scenario 2: A user wants a light-hearted, cartoonish feel. On the input form they select "Funny" as the tone and "Comic Book" as the art style instead of their previous choices. When they submit, the system runs the whole pipeline again (outline, story and images) with the new choices, producing a more humorous narrative and a classic comic-book look. Users can repeat this until the mood and visual style match their vision.

Scenario 3: After reviewing the on-screen preview, the user decides to download the comic. The system has already handled the following steps during generation:

1. **Layout Binding:** `layout_builder.py` assembles each panel's title, image, scene description, caption, narration and dialogue into a structured layout.

2. **PDF Generation:** `exporters.py` compiles the comic into a PDF using **FPDF**, placing each panel's image and text on its own page and saving the file with a timestamped name such as `comic_20260928_192148.pdf`.
3. **Export Success:** The "Download Comic PDF" button on the preview page opens the exported file, and the `/export-success` page confirms that the comic was exported and offers a link to create another one. This lets users save, print or share their comic in PDF format.

Pre-requisites

1. **FastAPI Framework Knowledge:** FastAPI documentation
2. **Uvicorn ASGI Server:** Uvicorn docs
3. **Python Programming Proficiency:** Python official docs
4. **HTML, CSS and Jinja2 Template Skills:** W3Schools HTML/CSS and Jinja2 tutorials
5. **Google Gemini API Familiarity:** Google Generative AI documentation
6. **Hugging Face Inference and FLUX Image Models:** Hugging Face Hub and Diffusers documentation
7. **PyTorch Basics:** PyTorch documentation
8. **PDF Generation with FPDF:** FPDF2 documentation
9. **Version Control with Git:** Git documentation
10. **Python Package and Environment Setup (pip, virtualenv or conda):** Virtualenv guide

Project Workflow

Milestone 1. Model selection and Architecture

- Activity 1.1. Research and Select the Appropriate Generative AI Model
- Activity 1.2. Define the architecture of the application
- Activity 1.3. Set up the development Environment

Milestone 2. Core functionalities Development

- Activity 2.1. Develop the core functionalities
- Activity 2.2. Implement the FastAPI Backend to Manage Routing and User Input Processing

Milestone 3. routes.py Development

- Activity 3.1. Writing the Main Application Logic in routes.py

Milestone 4. Frontend Development

- Activity 4.1. Designing and Developing User Interface
- Activity 4.2. Creating Dynamic Templates with FastAPI's Jinja2

Milestone 5. Deployment

- Activity 5.1. Preparing the Application for Local Deployment
- Activity 5.2. Testing and Verifying Local Deployment

Milestone 1. Model selection and Architecture

This milestone selects the generative AI models for ComicCraft. The chosen models must produce comic storylines, character dialogues, scene descriptions and comic-style illustrations that match the application's objectives.

Activity 1.1: Research and Select the Appropriate Generative AI Model

Understand the Project Requirements

The application must generate three kinds of content from a custom story prompt:

- Structured comic outlines (panel-by-panel)
- Full comic narration, captions and character dialogues
- Visual comic-style illustrations for each panel

Model Evaluation

The candidates were Google's Gemini models (Flash variants for fast structured output, larger variants for detailed creative text) and image models served through Hugging Face. They were compared on:

- Response quality for creative writing tasks (narration and dialogues)
- Prompt adherence and richness of detail for scene descriptions
- Speed and resolution of image generation
- Model stability, available documentation and community support

Practical testing with different prompts assessed each model's storytelling and image generation performance.

Final Model Selection

- **Gemini Flash family (`gemini_flash.py`):** generates the structured five-panel comic outline. The code tries a prioritized list of Flash models (`gemini-flash-lite-latest`, `gemini-flash-latest`, `gemini-3.1-flash-lite` and newer Flash releases), retrying twice per model and moving to the next model if one fails.

- **Gemini text model (`gemini_pro.py`):** writes the narration, caption and dialogue for every panel. It is configured with `gemini-3.1-flash-lite`.
- **FLUX.1-schnell (`black-forest-labs/FLUX.1-schnell`):** generates a 1024 x 1024 comic-style illustration for each panel through the Hugging Face `InferenceClient`.

This combination delivers rich, creative storytelling alongside vivid comic panels in an automated, efficient manner.

Activity 1.2: Define the architecture of the application

The architecture of ComicCraft has three primary components:

- **Frontend** (HTML, CSS, Jinja2)
- **Backend** (FastAPI application)
- **AI Integration** (Google Gemini API and Hugging Face image inference)

The frontend captures user inputs and displays generated comics, the backend handles data processing and routing, and the AI services generate story content and illustrations.

Frontend Responsibilities

- Users interact with a simple web interface built with HTML, CSS and Jinja2 templates.
- Key user input fields:
  - **Story Prompt:** main idea for the comic
  - **Character Name:** hero of the comic
  - **Setting:** location (Forest, School, City, Space)
  - **Story Tone:** mood of the story (Funny, Dramatic, Light-hearted, Poetic)
  - **Art Style:** visual style preference (Anime, Comic Book, Pixel Art, Realistic)
- On form submission, the data is sent to the FastAPI backend via a POST request for processing.

Templates used:

- `index.html`: for user input collection
- `comic_preview.html`: to display the generated comic
- `export_success.html`: to show a success message after comic export

Backend Responsibilities

- FastAPI manages server-side operations and route handling.
- Key backend tasks:
  - Receive form data from the frontend
  - Call AI models to generate panel outlines, story narration and illustrations

    - Organize and build the complete comic layout
    - Export the comic to a downloadable PDF
- Routes are defined in `routes.py` to handle the homepage, form-based comic generation and the export confirmation page.

AI Integration Points

- The **Gemini Flash** module is called first to generate the structured five-panel comic outline from the user's story prompt, character, setting, tone and art style.
- The **Gemini text model** module is called next to create narration, captions and character dialogue from the outline.
- **FLUX.1-schnell** generates the comic illustration for each panel from the AI-generated image prompts.
- The backend receives the AI responses, formats the text and images, and passes them to the frontend templates for preview and export.

All AI model interactions happen dynamically during comic generation, so every comic is unique and customized to the user's input.

Activity 1.3: Set up the development Environment

Install Python & Pip: Ensure Python is installed on your system along with pip, which manages the project dependencies.

Create Virtual Environment: This keeps project-specific packages from conflicting with system-wide installations.

```
python -m venv comiccraft-env
comiccraft-env\Scripts\activate   # For Windows
```

Install Required Libraries:

```
pip install fastapi uvicorn jinja2 python-multipart google-genai python-
dotenv diffusers transformers torch accelerate Pillow fpdf2
```

Run Server:

```
uvicorn app.main:app --reload
```

Visit http://127.0.0.1:8000 for the app and http://127.0.0.1:8000/docs for API testing.

Set Up Project Structure: Create the initial project directory structure for ComicCraft:

```
comiccraft/
|
|-- app/
|   |-- __init__.py
|   |-- main.py
|   |-- routes.py
|   |-- gemini_flash.py
|   |-- gemini_pro.py
|   |-- image_generator.py
|   |-- layout_builder.py
|   |-- exporters.py
|
|-- templates/
|   |-- index.html
|   |-- comic_preview.html
|   |-- export_success.html
|
|-- static/
|   |-- panels/      # Generated panel images
|   |-- exports/     # Exported comic PDFs
|
|-- .env             # Environment variables (API keys)
|-- requirements.txt # Python dependencies list
```

Milestone 2. Core functionalities Development

Activity 2.1: Develop the core functionalities

Generate Structured Comic Panel Outline

Function: `generate_outline()` · File: `gemini_flash.py`

- Uses the Gemini Flash models to generate a structured five-panel comic outline from the user's story prompt, character name, setting, tone and art style.
- Each panel contains a panel number, title, scene description and an image generation prompt.
- The output is a list of dictionaries, one for each comic panel.
- Markdown code fences are stripped from the response before it is parsed as JSON. If a model fails, the function retries once and then falls back to the next model in the `MODELS` list.

```
def generate_outline(story_prompt, character_name, setting, tone, art_style):

    prompt = f"""
Create a 5-panel comic outline.

Story:
{story_prompt}

Character:
{character_name}

Setting:
{setting}

Tone:
{tone}

Art style:
{art_style}

Return ONLY valid JSON.

Format:

[
    {{
        "panel_number": 1,
        "title": "Panel title",
        "scene_description": "Scene description",
        "image_prompt": "Image prompt"
    }}
]
"""

    for model in MODELS:
        for attempt in range(2):
            try:
                response = client.models.generate_content(
                    model=model,
                    contents=prompt
                )
                text = response.text.strip()

                # Remove JSON code fences
                if text.startswith("```json"):
                    text = text[7:]
```

```
                if text.startswith("```"):
                    text = text[3:]
                if text.endswith("```"):
                    text = text[:-3]

                return json.loads(text.strip())

            except Exception as e:
                if attempt == 0:
                    time.sleep(3)

    raise RuntimeError("All Gemini text models failed.")
```

Generate Comic Story Narration and Dialogue

Function: `generate_story()` · File: `gemini_pro.py`

- Takes the panel outline and asks Gemini to expand it into a full comic-style story.
- Creates a narration, a caption and character dialogue for every panel.
- The output is a JSON string containing the story text for all panels.

```
def generate_story(outline):

    prompt = f"""
Create a comic story from this 5-panel outline.

{outline}

For every panel create:

- narration
- caption
- dialogue

Return ONLY valid JSON.

Format:

[
    {{
        "panel_number": 1,
        "narration": "Narration",
        "caption": "Caption",
        "dialogue": "Dialogue"
    }}
]
```

```
"""

    response = client.models.generate_content(
        model="gemini-3.1-flash-lite",
        contents=prompt
    )

    text = response.text.strip()

    if text.startswith("```json"):
        text = text[7:]
    if text.startswith("```"):
        text = text[3:]
    if text.endswith("```"):
        text = text[:-3]

    return text.strip()
```

Generate Comic Illustrations from Prompts

Function: `generate_image()` · File: `image_generator.py`

- Wraps the panel's image prompt in a comic-book style instruction and asks FLUX.1-schnell to create the illustration. The instruction tells the model not to draw any text, captions or speech bubbles.
- Saves the generated image as `panel_<number>.png` in the `static/panels` directory.
- Returns the web path to the saved image.

```
def generate_image(image_prompt, panel_number):

    prompt = f"""
Create a high-quality colorful comic-book illustration.

Scene:
{image_prompt}

Style:
Comic book art, colorful, detailed,
adventure atmosphere, cinematic lighting.

Do not include any text,
captions, speech bubbles,
or words inside the image.
"""

    image = client.text_to_image(
```

```
        prompt,
        model="black-forest-labs/FLUX.1-schnell"
    )

    filename = f"panel_{panel_number}.png"
    filepath = os.path.join(OUTPUT_DIR, filename)
    image.save(filepath)

    return f"/static/panels/{filename}"
```

Organize Comic Panels into Layout Structure

Function: `build_comic_layout()` · File: `layout_builder.py`

- Organizes the generated images and the full comic story into a structured layout.
- Matches each panel's image and outline with its narration, caption and dialogue by panel number.
- The output is a list of dictionaries containing the panel number, title, scene description, image path, narration, caption and dialogue.

```
def build_comic_layout(panels, story):

    story_data = json.loads(story)

    story_by_panel = {}
    for item in story_data:
        story_by_panel[item["panel_number"]] = item

    layout = []

    for panel in panels:
        number = panel["panel_number"]
        story_info = story_by_panel.get(number, {})

        layout.append({
            "panel_number": number,
            "title": panel["title"],
            "scene_description": panel["scene_description"],
            "image": panel["image"],
            "narration": story_info.get("narration", ""),
            "caption": story_info.get("caption", ""),
            "dialogue": story_info.get("dialogue", "")
        })

    return layout
```

Export Comic Panel and Story into PDF

Function: `save_pdf()` · File: `exporters.py`

- Compiles the full comic into a multi-page **PDF file** using the FPDF library.
- Each panel's title, image, scene description, caption, narration and dialogue are placed on a separate page.
- The PDF is saved into the `static/exports` folder with a timestamped filename, and its web path is returned.

```
def save_pdf(layout):

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"comic_{timestamp}.pdf"
    filepath = os.path.join(OUTPUT_DIR, filename)

    pdf = FPDF()
    pdf.set_auto_page_break(auto=True, margin=15)

    for panel in layout:
        pdf.add_page()

        # Panel title
        pdf.set_font("Arial", "B", 18)
        pdf.cell(0, 12, clean_text(
            f"Panel {panel['panel_number']}: {panel['title']}"), ln=True)
        pdf.ln(5)

        # Panel image
        image_path = panel["image"].replace("/static/", "static/")
        if os.path.exists(image_path):
            pdf.image(image_path, x=15, w=180)
        pdf.ln(8)

        # Scene description, caption, narration and dialogue
        pdf.set_font("Arial", "I", 11)
        pdf.multi_cell(0, 7, clean_text(panel["scene_description"]))
        pdf.ln(5)
        pdf.set_font("Arial", "B", 11)
        pdf.multi_cell(0, 7, "Caption: " + clean_text(panel["caption"]))
        pdf.ln(3)
        pdf.set_font("Arial", "", 11)
        pdf.multi_cell(0, 7, "Narration: " + clean_text(panel["narration"]))
        pdf.ln(3)
```

```
        pdf.multi_cell(0, 7, "Dialogue: " + clean_text(panel["dialogue"]))

    pdf.output(filepath)
    return f"/static/exports/{filename}"
```

Activity 2.2: Implement the FastAPI Backend to Manage Routing and User Input Processing

Define Routes in FastAPI

All routing logic is handled in `routes.py`. Each route is linked to the backend functions developed in Activity 2.1, and `main.py` mounts the `/static` folder and includes the router.

Process User Input

- **HTML Form:** created in `index.html`
- **Users enter:** story prompt, main character name, setting, tone and art style
- **FastAPI captures inputs** using `Form(...)` parameters from the request

Integrate Gemini API Calls

Each route handler connects directly to the AI-powered function modules:

- `/generate` →
  - Calls `generate_outline()` to create panel outlines from the user's inputs
  - Calls `generate_story()` to expand the outline into narration, captions and dialogue
  - Calls `generate_image()` for each panel to create comic-style images
  - Calls `build_comic_layout()` to organize the final layout
  - Calls `save_pdf()` to compile and export the complete comic into a PDF

All AI-generated responses (text and images) are processed and injected into the Jinja2 frontend templates (`comic_preview.html`, `export_success.html`) for a seamless user experience.

Milestone 3. routes.py Development

This milestone sets up the FastAPI routes for the app, handles user inputs, integrates the AI generation workflow, and connects backend processing to the frontend templates. It forms a stable foundation for processing prompts, generating comics and giving users a smooth experience.

Activity 3.1: Writing the Main Application Logic in routes.py

Defining Core routes in routes.py

Separate routes were established for each of ComicCraft's core functionalities:

- `/` → Loads the **homepage** (`index.html`) where users submit their story details.
- `/generate` → Handles **form submission**, processes the input using AI, generates the comic panels and returns the **comic preview** page.
- `/export-success` → Displays a **success confirmation page** after the comic is exported.

Set up Route Handlers for Each Feature

For `/generate`, user inputs are captured using `Form(...)` parameters for:

- Story prompt (`story_prompt`)
- Character name (`character_name`)
- Setting (`setting`)
- Story tone (`tone`)
- Art style (`art_style`)

The form values are passed into the backend functions (`generate_outline`, `generate_story`, `generate_image`, `build_comic_layout`, `save_pdf`) so the AI processing reflects the user's details.

Integrate AI workflows in each Route

Within `/generate`, the core AI functionalities are integrated in five steps:

1. `generate_outline()` creates the five-panel structured story.
2. `generate_story()` writes the narration, captions and dialogues.
3. `generate_image()` is called iteratively to generate an image for each panel.
4. `build_comic_layout()` structures the final panel layout.
5. `save_pdf()` compiles the story and illustrations into an exportable PDF.

Each AI-generated output is processed and passed to the HTML template (`comic_preview.html`), together with the PDF path for the download button.

Comic Generation Route:

```
@router.post("/generate", response_class=HTMLResponse)
async def generate_comic(
    request: Request,
    story_prompt: str = Form(...),
    character_name: str = Form(...),
```

```
    setting: str = Form(...),
    tone: str = Form(...),
    art_style: str = Form(...)
):

    # Step 1
    outline = generate_outline(
        story_prompt, character_name, setting, tone, art_style
    )

    # Step 2
    story = generate_story(outline)

    # Step 3
    panels = []
    for panel in outline:
        image_path = generate_image(
            panel["image_prompt"], panel["panel_number"]
        )
        panels.append({
            "panel_number": panel["panel_number"],
            "title": panel["title"],
            "scene_description": panel["scene_description"],
            "image": image_path
        })

    # Step 4
    layout = build_comic_layout(panels, story)

    # Step 5
    pdf_path = save_pdf(layout)

    return templates.TemplateResponse(
        request=request,
        name="comic_preview.html",
        context={"layout": layout, "pdf_path": pdf_path}
    )
```

Home Route:

```
@router.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse(
        request=request,
        name="index.html",
        context={}
    )
```

Export Success Route:

```
@router.get("/export-success", response_class=HTMLResponse)
async def export_success(request: Request):
    return templates.TemplateResponse(
        request=request,
        name="export_success.html",
        context={}
    )
```

Milestone 4. Frontend Development

This milestone builds clean, responsive web pages using **HTML, CSS and Jinja2 templating**. Each template is rendered dynamically through FastAPI, so users can submit inputs, view the generated comic panel by panel and confirm comic export actions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure

`index.html` is the main entry point for user input. Its form captures:

- Story Prompt
- Main Character Name
- Setting
- Story Tone
- Art Style

Each field is labeled and grouped inside a single form. Navigation is minimal to maintain focus, with a direct flow from input to comic generation.

Design a Responsive Layout Using CSS

- CSS is embedded directly within each HTML file.
- Layouts use a centered container with a maximum width (700 px for the form, 800 px for the comic panels) so content stays readable on different screen sizes.

- Consistent button styling with a hover effect is used.
- Input fields and comic panels use padding and rounded corners for a clean, modern look.
- A light grey background with white cards keeps the pages easy to read.

Create Separate Pages for Each Core Functionality

Three Jinja2-powered HTML templates were created under the `templates` folder:

- **`index.html`**: a form where users enter the story prompt, character name, setting, tone and preferred art style. It submits the data to the `/generate` FastAPI route via POST.
- **`comic_preview.html`**: displays the AI-generated comic panel by panel. Each panel includes:
  - Title
  - Scene illustration (image)
  - Scene description (in italics)
  - Caption, narration and dialogue text

  This lets users review the full comic storyline sequentially, and it provides the "Download Comic PDF" and "Create Another Comic" buttons.
- **`export_success.html`**: displays a success message after the comic is exported and encourages users to return and create a new comic.

Templates are modular and separated by functionality, and the UI moves smoothly between comic creation and comic review.

Activity 4.2: Creating Dynamic Templates with FastAPI's Jinja2

Integrate Jinja2 Templating for Dynamic Content Rendering

FastAPI's `Jinja2Templates` is used for template rendering:

```
from fastapi.templating import Jinja2Templates

templates = Jinja2Templates(directory="templates")
```

Each FastAPI route uses `TemplateResponse` to render HTML templates based on the user's interaction:

- `/` route renders `index.html`
- `/generate` route renders `comic_preview.html`
- `/export-success` route renders `export_success.html`

This dynamic binding ensures that the comic content (layout, images, text) updates automatically after user input.

Bind Backend Data to HTML Templates

➤ index.html

- Form elements use `name="..."` attributes that map directly to the backend `Form(...)` parameters in FastAPI (`story_prompt` , `character_name` , `setting` , `tone` , `art_style`).
- The form submits data via POST to `/generate` .

➤ comic_preview.html

- Dynamically displays:
  - Panel number and title
  - Image generated for the panel
  - Scene description (in italics)
  - Caption, narration and dialogue
- Loops through the `layout` list using the Jinja2 `{% for panel in layout %}` syntax.
- Uses the `pdf_path` variable as the target of the download button.

```
{% for panel in layout %}

<div class="panel">
    <h2>Panel {{ panel.panel_number }}: {{ panel.title }}</h2>

    <img src="{{ panel.image }}" alt="Comic Panel">

    <p class="scene">{{ panel.scene_description }}</p>

    <p><strong>Caption:</strong> {{ panel.caption }}</p>
    <p><strong>Narration:</strong> {{ panel.narration }}</p>

    <div class="dialogue">
        <strong>Dialogue:</strong> {{ panel.dialogue }}
    </div>
</div>

{% endfor %}

<a class="download" href="{{ pdf_path }}" target="_blank">
    Download Comic PDF
</a>
```

➤ export_success.html

- Displays a confirmation message that the comic has been exported.

- Provides a button for returning to the homepage to create another comic.

The frontend of ComicCraft is integrated with FastAPI, renders AI-generated comics dynamically, and provides a smooth user flow from prompt input to comic export.